

广度优先搜索

广度优先搜索，即按层拓展，就是优先拓展离起点最近的点，然后一步一步拓展离当前点最近的点，最后达到目标状态。因为广度优先搜索每次都是搜索所有离当前点最近的点，所以离起点远的点一定比离以前近的点先拓展，所以广度优先搜索第一次拓展到一个点时就是起点到该点的最短路径。

先被拓展的点优先去拓展其他点，所以广度优先搜索是通过一个队列来实现的，每搜到一个点就是起点到该点的最短路径。

广度优先搜索

```
#include<queue>
queue<node> q;
先特判起点
q.push(起点)
while(!q.empty())
{
    node t=q.front();
    q.pop();
    枚举下一个点的所有情况
    判断是否可行
    判断是否是终点
    该点入队
}
```

双端队列 BFS

因为广度优先搜索拓展到一个点就是起点到该点的最短路径，所以广搜必须要满足的条件是任意两点之间的路径相等。否则不可以使用广搜。

当然也有一种特殊的情况，如果是路径权值要么为0，要么都为 k (定值)，那么也是可以用广搜来做的，只是做法有一点不一样，这种情况采用的是双端队列广搜，即队首不仅可以进入，还可以出去。

如果拓展到权值为0的路径，那么说明该点和队首元素处于同一层，所以我们就可以在队首拓展。如果拓展到权值为 k 的路径，那么说明该点位于下一层，就放在队尾。注意head的初始值不能为0，否则会越界。

双端队列BFS

```
,
int head=250000; // 因为要放入队首,
// 所以为了防止数组越界, 队首从n*n开始
int tail=250000;
...
while(head<tail){
    for(int k=0;k<4;k++){
        head++
        ...
        if(map[tx][ty]==map[tmp.x][tmp.y])
        {
            head--; // 注意入队首和队尾的写法
            q[head].x=tx;
            q[head].y=ty;
        }
        else
        {
            q[tail].x=tx;
            q[tail].y=ty;
            tail++;
        }
    }
}
```



广度优先搜索

对于广搜而言，比较重要的一个地方是状态的判断，一般情况下到达一个点就是起点到该点的最短路。但是有的情况下并不是这样，因为仅仅是坐标位置的状态不唯一，有可能还有物品自身的状态。只有该标记完全能表示唯一的状态的时候才表示重复。



机器人搬重物

题目描述

[展开](#)

机器人移动学会 (RMI) 现在正尝试用机器人搬运物品。机器人的形状是一个直径1.6米的球。在试验阶段，机器人被用于在一个储藏室中搬运货物。储藏室是一个 $N \times M$ 的网格，有些格子为不可移动的障碍。机器人的中心总是在格点上，当然，机器人必须在最短的时间内把物品搬运到指定的地方。机器人接受的指令有：向前移动1步 (Creep)；向前移动2步 (Walk)；向前移动3步 (Run)；向左转 (Left)；向右转 (Right)。每个指令所需要的时间为1秒。请你计算一下机器人完成任务所需的最少时间。

机器人搬重物

因为要标记当前机器人所处的状态以免重复，如果只是标记`book[x][y]`，那么很明显是不对的，因为虽然位置固定了，但是在该位置的方向我们却不知道，而且不同的方向移动的位置也不一样，所以我们还需要标记一维来表示方向。并且该题移动也和以前不一样，不能直接一个循环就表示出来了，所以要分不同的情况讨论。



Bloxorz

§Bloxorz是一个风靡世界的小游戏。Bloxorz的地图是一个N行M列的矩阵，每个位置可能是硬地（用.表示）、易碎地面（用E表示）、禁地（用#表示）、起点（用X表示）或终点（用O表示）。你的任务是操作一个1*1*2的长方体。这个长方体在地面上有两种放置形式，“立”在地面上（1*1的面接触地面）或者“躺”在地面上（1*2的面接触地面）。在每一步操作中，可以按上下左右四个键之一。按下之后，长方体向对应的方向沿着棱滚动90度。任意时刻，长方体不能有任何部位接触禁地（否则就会掉下去），并且不能立在易碎地面上（否则会因为压强太大掉下去）。X标识长方体的起始位置，地图上可能有一个X或者两个相邻的X。地图上唯一的一个O标识目标位置。求把长方体移动到目标位置（即立在O上）所需要的最少步数。如果无解，输出Impossible。在移动过程中，X和O标识的位置都可以看作是硬地被利用， $3 \leq N, M \leq 500$ 。





Bloxorz

该题的状态表示也差不多，仅仅 $[x, y]$ 表示的状态不唯一，在 (x, y) 这个位置是竖着？横躺着？竖躺着？我们都不清楚，状态不清楚，肯定也就没有办法继续写下去。所以我们还要开一维来维护一个当前状态。注意状态一定要清楚，比如该题躺着要分横躺和竖躺，这两种状态是不一样的，必须要区分开来。



骰子

题目描述

一个 $N \times M$ 的矩形迷宫左上角的格子里放了一个骰子。骰子的展开图如图所示。其中，1 对应的是上侧面（就是当你俯视这个骰子的时候正对你的那个面），2 对应的是左侧面。骰子的每个面的大小和迷宫的每个小格的大小相同。

骰子可以在迷宫的棋盘上翻动到相邻的格子。但是不能翻出迷宫的边界。

你的任务是找到一条通路，使得骰子能从迷宫的左上角翻到右下角，且到达目的地时的上侧面上的数值最大。骰子可以经过同一个格子若干次。

骰子

同样的，如果要确定一个骰子的位置状态，我们不仅仅要知道这个骰子此时所处的位置，还要知道当前骰子的状态，因为骰子状态不同，就算在同一个位置那么答案也是不同的。所以我们还要想办法固定骰子的状态，如果只固定一个面，我们会发现骰子是可以以这个点为轴旋转的，所以我们需要固定骰子的两个相邻面。同样，移动只能上下左右，我们只需要知道骰子三个相邻面的点数就可以推出每次变换的新位置的点数。

魔法王国

题目描述

从古代开始，魔法王国就有一个由双向魔法门构成的网络。每一个魔法门以魔法连接一对城市，允许他们进行魔法交流和旅行。被魔法门连接起来的城市被称为相邻的。

Albert 王子和 Betty 公主住在相邻的城市。所以，在他们的孩提时代，Albert 和 Betty 经常通过魔法通讯球保持联系，魔法通讯球是通过连接城市的魔法门工作的。

Albert 和 Betty 相爱了。他们爱得如此深以至不能在没有对方时生活一分钟。他们总是把魔法球带在身边，从而，他们可以在任何时候与对方说话。他们的爱情有一个奇怪的事情——他们从没见过对方，甚至他们害怕同时在一个城市。人们说，是魔法球影响了他们。

对于 Albert 和 Betty 来说，在王国里旅行是一件复杂的事情。他们必须通过魔法门旅行，这对于皇家而言也是非常昂贵的。他们可以同时使用不同的魔法门移动到不同的城市，也可以一方使用魔法门移动到一个城市，另一方呆在一个城市。在任何时候他们必须在相邻的城市。他们不能同时使用同一个魔法门移动。

写一个程序来帮助 Albert 和 Betty 从一对城市到另外一对城市。必须找到一个最便宜的方案——他们通过魔法门移动的次数最少。如果他们同时使用魔法门，计做两次移动。

大逃亡

题目描述

给出数字 N ($1 \leq N \leq 10000$) , X ($1 \leq x \leq 1000$) , Y ($1 \leq Y \leq 1000$) ,代表有 N 个敌人分布一个 X 行 Y 列的矩阵上,矩形的行号从0到 $X-1$,列号从0到 $Y-1$ 再给出四个数字 x_1, y_1, x_2, y_2 ,代表你要从点 (x_1, y_1) 移到 (x_2, y_2) 。在移动的过程中你当然希望离敌人的距离的最小值最大化,现在请求出这个值最大可以为多少,以及在这个前提下,你最少要走多少步才可以回到目标点。注意这里距离的定义为两点的曼哈顿距离,即某两个点的坐标分为 $(a, b), (c, d)$,那么它们的距离为 $|a-c|+|b-d|$ 。

大逃亡

题目中有很明显的最大值最小，所以就是二分答案，然后跑一遍dfs验证，但是验证的时候要注意，当我们假设了答案 x 之后，那么到该点曼哈顿距离小于 x 的点都走不了，那么我们该如何标记呢？

如果把所有小于等于 x 的点都标记，那么时间是过不了的，所以我们实际上只需要标记最外圈就可以了。如果我们在最外圈外面走，那么一定走不进来。

该题还有一个坑点是，如果你这样做了，那么你二分的答案范围如果太大，那么就会导致你可能会在一个圈内移动，而圈内是没被标记的，所以会导致答案错误

广搜求方案数

广搜，第一次搜到这个点就是到这个点的最短路，但是处于这一层的点路径长度都是相同的，那么也会有其他点可以拓展到终点，比如出于第 $K-1$ 层的点 x 可以拓展到终点，处于第 $K-1$ 层的点 y 可以拓展到终点，处于第 $K-1$ 层的点 z 也可以拓展到终点，那么这三种拓展方案都是最短路的方案数。

那么从起点到终点的最短路径方案数就是3吗？

起点	
	A
B	终点

从起点到终点的最短路径方案数有4种，但是能够到达终点的点明显只有两个点，那么计算出的答案最多也就是2，所以明显是错误的。

那么错误的原因是什么呢？

广搜求方案数

既然从起点到终点的最短路不止一条，那么起点到任意一个点的最短路也很有可能不止一条，但是我们在累加的时候，每个能到达终点的点都是+1，那就意味着起点到x, y, z的最短路无论如何都只有一条，那这明显是不对的。

比如起点到A点的最短路有3条，到B点的最短路有1条，加的时候应该是3+1，而不是1+1，所以我们如果要知道起点到终点的最短路径方案数，还必须知道起点到任意一点的最短路的方案数。

所以重新开一个数组num[x][y]即可。同时还需要一个数组来记录起点到任意一点的层数，层数相同时，累加方案数。

起点	
	A
B	终点

广搜求方案数

```
if(book[tx][ty]==0)
{
    if(tx==X&&ty==Y)//找到了终点
    {
        flag=1;//这一层找完就可以不用找了
    }
    book[tx][ty]=1;
    q.push(node(tx,ty,tmp.step+1));
    num[tx][ty]=num[tmp.x][tmp.y];//继承上一个的方案数
    step[tx][ty]=tmp.step+1;
}
else if(book[tx][ty]==1&&step[tx][ty]==tmp.step+1)
{
    num[tx][ty]+=num[tmp.x][tmp.y];
    //有多个点可以到达，一定要累加
}
```



广搜求具体方案

广搜求具体方案数也是一样的原理，广搜中，如果可以到达这个点，那么一定就是最短路径，并且每个点只会被更新一次，所以直接记录前驱就行，根本不需要更新。当更新完终点之后再顺着前驱一直寻找到起点得到的路径就是最终的答案。



双向搜索

在一道题中，如果已知初始状态和最终状态，比如已知起点和终点，我们可以采用双向搜索，即同时从起点和终点出发往中间搜索，每次搜索判断两颗搜索树是否有交叉，这样搜索深度就会减半，时间复杂度会大大减小

比如，每增加一层可能性 $\times 2$ ，起点到终点的最短路为 l ，直接搜索的可能性为 2^l ，如果双向搜索，搜索的可能性为 $2^{l/2} \times 2 \times 2$ ，时间大大减少了，但是对代码的要求比较高，每次增加一层都需要判断两颗搜索树是否有交点。双向搜索有很多种情况，并不只是同时从起点和终点往中间搜索，即双向bfs，其实dfs也可以双线，先搜前半段，再搜索后判断，然后前后相加。

Nightmare II

题意：

给定 $n \times m$ 的地图，现在地图上一些格子是墙，人不能通过。地图上有一个男孩和一个女孩，男孩一次能跑 3 格，女孩一次能跑 1 格。

现在地图上有两个幽灵，每个格子的幽灵每秒钟可以扩散到曼哈顿距离不超过 2 的所有格子，被扩散的格子都会变成有幽灵的，墙不能阻挡幽灵，它们会一直扩散直到遍布整个地图。

假设每个单位时间内，都是幽灵先于男孩女孩移动。现在问你男孩和女孩在不碰到幽灵的前提下，能不能会和在一个格子。

该题就是一道裸的双向bfs题，需要同时从男孩和女孩的位置开始搜索，如果双方同时扩展到了一个位置，并且该位置是合法的就是最终的结果。

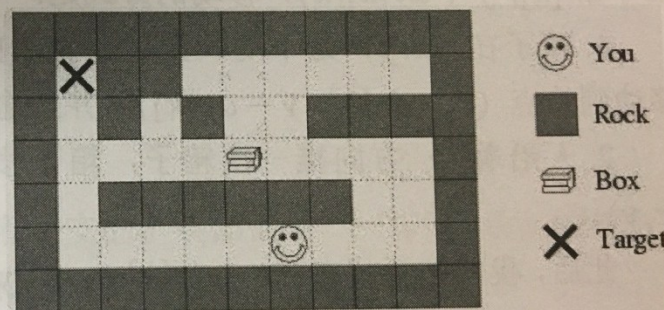
P0J 2475 推箱子

【例题】Pushing Boxes POJ1475

推箱子游戏相信大家都不陌生，在本题中，你将控制一个人把 1 个箱子推到目的地。给定一张 N 行 M 列的地图，用字符“.”表示空地，字符“#”表示墙，字符“S”表示人的起始位置，字符“B”表示箱子的起始位置，字符“T”表示箱子的目标位置。求一种移动方案，使箱子移动的次数最少，在此基础上再让人移动的总步数最少。方案中使用大写的“EWSN”（东西南北）表示箱子的推动，使用小写的“ewsn”（东西南北）表示人的移动。 $N, M \leq 20$ 。

样例输入： 7 11

```
#####  
#T##.....#  
#.#.#..####  
#....B....#  
#.#####..  
#.....S...#  
#####
```



样例输出： eennwwWWWleeeeeesswwwwwnNN

推箱子游戏

该题和我们平时做的搜索的题又不一样，以前都是人自己走，现在变成了人推着箱子走，如果我们直接把人和箱子看做一个整体，那很明显是错误的，因为人和箱子的位置一旦固定，箱子的移动方向就固定了，所以我们只能分开考虑。题目要求是箱子移动步数最少，所以我们可以先看做是箱子自己可以动，求最优方案。然后再考虑箱子走的每一步，人的位置以及人该如何移动才能最快到达该位置，所以又是求从当前位置移动到目的地的最优方案，所以本题实际上就是bfs嵌套。

A* 算法

但是很多时候我们会发现，几乎是没有什么估计距离的，比如 n 个点的最短路，估计函数可以设置为0，但是对答案是没有影响的，那么A*的真正作用是干什么呢？

求解K短路。

给定一个 N 个点， M 条边的有向图，求从起点 S 到终点 T 的第 K 短路的长度，路径允许重复经过点或边，其中 $S, T \leq 1000, M \leq 100000, K \leq 1000$ 。

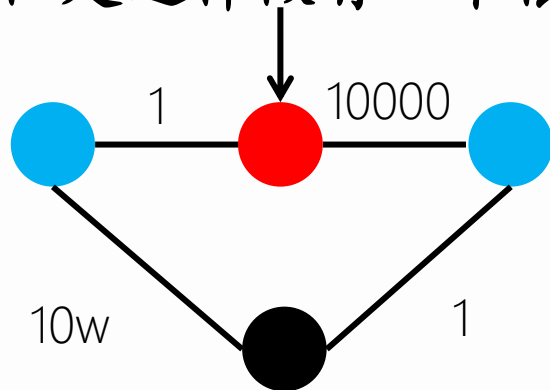
POJ 2449

A* 算法

我们以前求解的都是最短路径，当我们用广搜求解最短路径的时候，当一个点第一次被搜到的时候就是最短路径。实际上还有一点被我们忽略掉了，如果不标记，当某个点第二次被搜到的时候，就是次短路，所以实际上当一个点第k次被搜到的时候，就是k短路。这是根据广搜按层拓展的原理总结出来的。

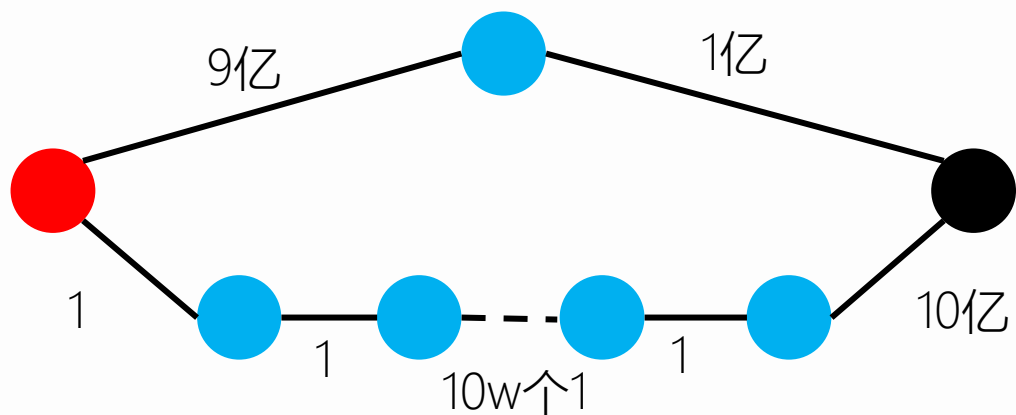
所以我们很容易想到第一种解法：直接跑dijkstra，当终点第k次出队的时候，路径的长度就是最终的答案。

但是这样做有一个很大的问题。



A* 算法

因为我们每次都是选择当前最优的点，并且一个点可以多次重复选择，那么最终就会导致，权值为1的点会被选择10000次之后才会去拓展权值为10000的点，可是万一权值为10000的点后面马上就是终点呢？那这样岂不是效率特别低？一直在走错误的道路。最坏时间复杂度为 $K * M \log M$ 。有的同学可能会说，优化一下不就好了，如果一个点出队了 k 次之后那么就完全没必要管他了，因为肯定不是 k 短路。好，我们来看一个极端的例子。



下面的一排点都会被拓展 k 次之后再拓展下一个点，效率超低！！！！

A* 算法

那如果我们考虑启发式搜索应该如何写估价函数呢？我们要求的是k短路，那么其中的某些边会被经过不止一次，那么对于到达的当前的每一个点 p ，我们知道 L_{sp} ，而我们需要估计的就是 G_{pt} ，并且估计值肯定小于等于该点到终点的k短路径中的距离。很容易想到，估价函数就写终点到任意一个点的最短路径就可以了。这就是一个优秀的估价函数。

接下来就是k短路的具体实现过程。

第一步：确定估价函数——最短路。

即求出终点到任意一个点的最短路径的距离，因为是有向图，所以我们要先建反向边，从终点开始跑最短路。得到一个dis数组。

A* 算法

第二步：确定估价函数——最短路。

跑A*算法。对于一个当前点 p ，记录 L_{sp} 的距离(起点到 p 点的实际距离)，记录 G_{pt} 的距离($dis[p]$)，每次从队列中取出 $L_{sp}+G_{pt}$ 最小的一个优先拓展，然后更新 L 和 G 的答案。

注意：

为了避免被卡死，任意一个点不能被拓展超过 k 次，假设一个点被拓展超过 k 次，那么这条路径就一定不是 k 短路，因为当前已经被拓展超过 k 次了。

这道题还有一个坑点是，如果起点=终点，不走是不算一条路径的。

代码。。。参考dijkstra



必做练习题

MSOJ	1668	电路维修
MSOJ	1454	魔法王国
洛谷	1126	机器人搬重物
MSOJ	1283	大逃亡
MSOJ	1212	小鼠走迷宫3
MSOJ	1337	骰子
MSOJ	1488	牛跑步

